

Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования «Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

09.03.01 Информатика и вычислительная техника

(направление подготовки/специальность)

Электронно-вычислительные машины, комплексы, системы и сети

(профиль/специализация)

очная

(форма обучения)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Тип практики Технологическая (проектно-технологическая) практика
в/на ООО «Веб-стрит»

ТЕМА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Изучение технологий потоковой обработки и проектирование легковесных систем хранения данных на языке C

Выполнил:

студент института информатики и вычислительной техники
группа ИВ-222

21.07.2025

_____/ Рудцких В.Е. /
(подпись) (ФИО)

Проверил:

Руководитель практики от СибГУТИ

21.07.2025

_____/ Ефимов А.В. /
(подпись) (ФИО)

отметка _____ «___» _____ 202_г.

Новосибирск 2025

НАПРАВЛЕНИЕ НА ПРАКТИКУ

На основе договора № 2024/97/66 от 15.02.2024 г.

федеральное государственное бюджетное образовательное учреждение высшего образования "Сибирский государственный университет телекоммуникаций и информатики" (СибГУТИ)

направляет студента 3 курса, института информатики и вычислительной техники, гр. ИВ-222

_____Рудцких Владислава Евгеньевича_____

(ФИО обучающегося)

для прохождения технологической (проектно-технологическая) практики с 04.06.2025 по 21.07.2025 в/на ООО «Веб-стрит»

МП Директор института _____/ _Приставка П.А._/

План-график проведения практики

Направление: 09.03.01 Информатика и вычислительная техника

(код, наименование направления (специальности))

Направленность (профиль)/ специализация: Электронно-вычислительные машины, комплексы, системы и сети

Объем практики: 360/10 часов/ЗЕ

Тип практики: Технологическая (проектно-технологическая) практика

Содержание практики:

Тема индивидуального задания практики: Изучение технологий потоковой обработки и проектирование легковесных систем хранения данных на языке C

Наименование видов деятельности в соответствии с рабочей программой практики	Дата (начало – окончание)
Общее ознакомление со структурным подразделением предприятия	04.06.2025 - 04.06.2025
Выдача задания на практику, деление студентов на группы, определение конкретной индивидуальной темы, формирование плана работ	04.06.2025 - 06.06.2025
Работа с библиотечными фондами структурного подразделения или предприятия, сбор и анализ материалов по теме практики	06.06.2025 - 09.06.2025
Разработка и подключение пользовательских заголовочных файлов (database.h) и реализация модульного кода (database.c, main.c). Проектирование и создание структуры данных на основе C-структур для хранения записей базы данных. Реализация CRUD-операций: добавление, поиск, обновление и удаление записей в файловой базе данных. Создание генератора тестовых данных для моделирования больших объёмов. Реализация потокового анализа больших данных (агрегация min/max/mean/count) без полной загрузки в память. Настройка и использование Makefile для автоматизации сборки проекта. Проведение тестирования и отладки: написание сценариев тестирования, использование Valgrind для поиска утечек памяти.	09.06.2025 - 17.07.2025
Анализ полученных результатов и произведённой работы, составление отчета по практике	17.07.2025 - 21.07.2025

Руководитель практики
от профильной организации
«_04_» __06__ 2025_г.

_____/ Гасенко М.С. /
(подпись) (ФИО)

Руководитель практики от СибГУТИ

/

«_02_» __06__ 2025_г.

_____ / _Ефимов А.В.

(подпись)

(ФИО)

Студент

«_03_» __06__ 2025_г.

_____ / Рудцких В.Е. /

(подпись)

(ФИО)

ДНЕВНИК РАБОТЫ

Дата/ период	Рабочее место и краткое содержание выполняемых работ	Отметка о выполнении (выполнено/ не выполнено)
04.06.2025 - 04.06.2025	Ознакомление со структурным подразделением предприятия	Выполнено
04.06.2025 - 06.06.2025	Получение задание на практику, определение конкретной индивидуальной темы, формирование плана работ	Выполнено
06.06.2025 - 09.06.2025	Работа с библиотечными фондами структурного подразделения, сбор и анализ материалов по теме практики	Выполнено
09.06.2025 - 16.06.2025	Поиск и анализ учебных материалов, технических статей и документации по низкоуровневому хранению данных в С	Выполнено
17.06.2025 - 20.06.2025	Тренировочные упражнения: управление динамической памятью, работа с файловыми дескрипторами, отладка простых структур	Выполнено
21.06.2025 - 27.06.2025	Проектирование логической схемы записи, описание интерфейса базы данных в database.h	Выполнено
28.06.2025 - 03.07.2025	Кодирование операций вставки и поиска (database_add, database_search), проверка корректности на малых наборах	Выполнено
04.07.2025	Автоматизация сборки: написание Makefile, настройка целей, флагов компиляции, устранение предупреждений компилятора	Выполнено
05.07.2025 - 08.07.2025	Реализация модификации и пометки удалённых записей (database_update, database_delete), отладка граничных случаев	Выполнено
09.07.2025 - 14.06.2025	Комплексное тестирование всех компонентов, динамический анализ Valgrind, исправление утечек и ошибок доступа к памяти	Выполнено
15.07.2025 - 16.07.2025	Систематизация результатов, оформление пояснительной записки, подготовка иллюстративного материала	Выполнено
17.07.2025	Согласование черновика отчёта с руководителем, корректировка замечаний	Выполнено
17.07.2025 - 21.07.2025	Обобщение итогов, финальная компоновка отчёта и сдача работы	Выполнено

КРАТКАЯ ХАРАКТЕРИСТИКА РАБОТЫ ПРАКТИКАНТА

Отзыв руководителя от предприятия о работе студента (выполнение программы практики, овладение производственными навыками, отношение к работе, трудовая дисциплина и др.)

Студент выполнил программу практики в полном объеме, изучил технологии:

Язык C в контексте системного программирования: работа с бинарными файлами и форматированным вводом-выводом, управление памятью в долгоживущих процессах, проектирование структур данных фиксированной длины для эффективного хранения.

Компилятор GCC и сборочная система Make: настройка многомодульной сборки с отдельной компиляцией объектных файлов, генерация отладочных и оптимизированных версий.

Система контроля версий Git и платформа GitLab: ведение истории изменений, ветвление для экспериментальных функций, слияние протестированных решений в основную ветку.

Интегрированная среда Visual Studio Code: настройка синтаксического анализа, быстрая навигация по модульному проекту, встроенные средства отладки, овладел

производственными навыками

Отношение к работе к работе ответственное.

Трудовая дисциплина: исполнительный и пунктуальный.

Студент выполнил программу практики в полном объеме, изучил технологии

Руководитель практики от предприятия: _____ / **Гасенко М.С.** /
М.П.

Отзыв руководителя от СибГУТИ о работе студента (выполнение программы практики, овладение производственными навыками, отношение к работе, трудовая дисциплина и др.)

С поставленными задачами студент справился в полном объеме.

Проявил любознательность и трудолюбие. В процессе практики студент

самостоятельно изучил необходимую теорию, выполнил и защитил практические задания в установленные сроки.

Компетенции	Уровень сформированности компетенций ¹
ПК-1 - Способен разрабатывать требования и проектировать программное обеспечение	

отметка о зачете (оценка) _____

Руководитель практики от СибГУТИ: _____/_Ефимов А.В._/

¹ Уровень сформированности компетенций: высокий, средний, низкий, не сформирована.

СОДЕРЖАНИЕ

Введение.....	9
Информация о компании.....	9
Производственная проблема.....	9
Обзор решения и используемых технологий.....	9
Подготовка рабочего места.....	10
Проектирование и разработка.....	10
Тестирование и отладка.....	12
Результаты внедрения.....	13
Выводы.....	14
Список использованных источников.....	15

Введение

Целью данной практики является закрепление теоретических знаний и получение практических навыков в области системного программирования на языке C, организации файлового ввода-вывода и потоковой обработки данных. В рамках индивидуального задания выполнена разработка компактной файловой базы данных с фиксированной структурой записей, а также утилит для генерации больших тестовых массивов и их аналитической обработки без загрузки всего объёма в оперативную память.

Для достижения цели нужно предпринять следующие:

- Углубить понимание механизмов работы с бинарными файлами и структурами в C;
- Освоить методику потоковой агрегации числовых данных;
- Разработать модульную архитектуру приложения с разделением интерфейса и реализации;
- Настроить автоматизированную сборку с помощью Makefile;
- Провести динамический анализ кода на предмет утечек памяти с использованием Valgrind.

Информация о компании

Практика проводилась в ООО «Веб-стрит» - Среда предприятия предполагает самостоятельную работу над проектами, обязательное модульное тестирование и применение инструментов непрерывной интеграции.

Производственная проблема

Для ряда внутренних сервисов требуется легковесное файловое хранилище, способное оперировать записями фиксированной длины без использования внешних СУБД. Дополнительно необходимы средства пакетной генерации тестовых данных и инструмент для быстрого статистического анализа логов большого объёма (сотни миллионов строк), который не потреблял бы память пропорционально размеру входного файла. Ключевые требования — минимальные накладные расходы, детерминированное потребление ресурсов и полное отсутствие утечек памяти.

Обзор решения и используемых технологий

Для решения поставленных задач были выбраны следующие инструменты:

- Язык C — прямой контроль памяти, высокая производительность операций ввода-вывода; Бинарные файлы и структуры фиксированного размера — быстрый доступ по смещению, компактное хранение;
- GCC и Make — компиляция с отладочными флагами, автоматическая сборка многомодульного проекта;
- Valgrind — динамический анализ кучи, проверка корректности доступа к памяти и файловым буферам

Для решения проблемы был выбран следующий подход:

- Логическое удаление записей (поле `is_deleted`) — исключение фрагментации файла при удалении;
- Поточковая обработка текстовых файлов — построчное чтение с немедленной агрегацией (`min`, `max`, `sum`, `count`, `mean`) без загрузки файла в память;

Подготовка рабочего места

Настройка окружения включала следующие шаги:

Установка компилятора `gcc` (версия 11.2) и утилиты `make`;

Установка `Valgrind` версии 3.18;

Клонирование корпоративного репозитория: `git clone <URL>`;

Создание отдельной ветки `feature/file-database` для изолированной разработки;

Проверка работоспособности инструментов на простейших примерах.

Проектирование и разработка

На начальном этапе была спроектирована структура записи `Entry`, включающая поля `id` (целочисленный идентификатор), `name` (строка фиксированной длины 50 символов), `age` (возраст) и `is_deleted` (флаг логического удаления). Выбор фиксированного размера структуры (`sizeof(Entry)` одинаков для всех записей) позволил организовать хранение в виде бинарного файла с последовательным размещением записей. Такой формат обеспечивает возможность прямого позиционирования по смещению (`fseek`), что критически важно для точечных операций обновления и удаления, а также упрощает восстановление данных при повреждении файла. Флаг `is_deleted` исключает необходимость физического удаления и перезаписи всего файла, что положительно сказывается на производительности при частых операциях удаления.

Интерфейс базы данных вынесен в заголовочный файл `database.h`, содержащий объявление структуры и прототипы публичных функций. Такое разделение интерфейса и реализации соответствует принципам модульного программирования и позволяет подключать готовый модуль к различным клиентским приложениям без повторения кода

В модуле `database.c` реализован полный набор CRUD-операций:

`database_open` и `database_close` — управляют временем жизни файла базы данных. Функция открытия в бинарном режиме чтения/записи (`r+b`) с автоматическим созданием файла при его отсутствии (`w+b`) гарантирует, что приложение всегда работает с корректным файловым дескриптором.

`database_add` — добавляет запись в конец файла, используя `fseek` на `SEEK_END` и последующую запись целой структуры. Вызов `fflush` принудительно сбрасывает буфер, обеспечивая немедленную фиксацию данных на диске.

`database_search` — выполняет последовательный перебор активных записей до первого совпадения идентификатора. Такой подход оправдан для файлов небольшого и среднего размера, а фиксированная длина записи делает возможной оптимизацию (например, прямой переход по индексу, если в будущем добавится индексный файл).

`database_update` — комбинирует поиск и точечную перезапись: после нахождения нужного `id` функция возвращает указатель на начало записи (`ftell`) и обновляет поля `name` и `age`, сохраняя неизменными идентификатор и флаг удаления.

`database_delete` — устанавливает флаг `is_deleted` в единицу, что исключает запись из результатов поиска и списка, но оставляет данные физически доступными для возможного последующего восстановления или компактизации.

`database_list_all` — вспомогательная функция для вывода всех активных записей в читаемом формате, что удобно при отладке и ручном контроле состояния базы.

Интерактивное приложение `main.c` связывает модуль базы данных с пользователем через консольное меню. Программа в бесконечном цикле ожидает ввод команды и вызывает соответствующую CRUD-функцию, запрашивая у пользователя необходимые параметры. Такой минималистичный интерфейс не требует графических библиотек и удобен для встраиваемых систем или удалённого администрирования через SSH.

Для массовой генерации тестовых данных разработана утилита `generate.c`. Она принимает имя выходного файла и количество строк, после чего с использованием генератора псевдослучайных чисел (`rand`) создаёт текстовый файл с числами в диапазоне `[-1000, 1000]`. Формат записи «одно число с плавающей точкой на строку» выбран для совместимости с утилитой потоковой обработки. Такой подход позволил получить файлы объёмом более 150 МБ (при 10 миллионах строк) и провести реалистичное нагрузочное тестирование.

Центральным элементом, демонстрирующим работу с большими данными, стала утилита `stream_stats.c`. Она построчно читает входной текстовый файл, преобразует каждую строку в число (`strtod`) и немедленно обновляет агрегатные метрики: количество, сумму, минимум, максимум. После завершения чтения вычисляется среднее арифметическое, и результат записывается в указанный выходной файл. Ключевая особенность архитектуры — постоянное потребление оперативной памяти. Буфер для чтения строки (`char line[256]`) и несколько переменных накапливающего типа — это весь объём данных, находящийся в памяти одновременно, независимо от размера входного файла. Подобный подход принципиален для обработки наборов данных, превышающих доступный объём ОЗУ, и характерен для систем, работающих с потоками данных (`data streaming`). В ходе реализации особое внимание уделено обработке ошибок: некорректные строки пропускаются с выводом предупреждения, а при возникновении ошибки ввода-вывода программа корректно завершается, освобождая все ресурсы.

Автоматизация сборки реализована с помощью `Makefile`. Файл содержит следующие цели:

`all` (по умолчанию) — сборка исполняемых файлов `database_app` и `test_database`;

`test_database` — программа модульного тестирования;

`generate` и `stream_stats` — вспомогательные утилиты;

clean — удаление объектных файлов и артефактов;

test — запуск модульного теста и автоматический прогон под Valgrind.

Для всех компилируемых модулей заданы флаги -Wall -Wextra -g -O0, что включает вывод всех предупреждений и отключает оптимизацию, облегчая пошаговую отладку в GDB и анализ Valgrind. Зависимости объектных файлов от заголовочного файла database.h явно прописаны, поэтому изменение интерфейса автоматически вызывает пересборку всех затронутых компонентов.

Модульное тестирование CRUD-операций вынесено в отдельный исполняемый файл test_database.c, что позволяет изолированно проверять корректность функций базы данных без участия пользовательского интерфейса. Тестовые сценарии включают в себя проверку добавления, поиска (в том числе несуществующего идентификатора), обновления и логического удаления. Использование макроса assert обеспечивает немедленную остановку с диагностикой при обнаружении несоответствия. Такой подход значительно ускорил цикл отладки и позволил быстро локализовать ошибки на ранних этапах.

Все компоненты проекта были реализованы исключительно с использованием стандартной библиотеки C (stdio, stdlib, string, assert), что гарантирует отсутствие внешних зависимостей и максимальную переносимость между POSIX-совместимыми системами. Итоговая кодовая база не содержит динамического выделения памяти (кроме внутренних структур библиотечных функций, которые корректно освобождаются при закрытии файлов), что было подтверждено проверкой Valgrind с опциями --leak-check=full --show-leak-kinds=all. Достигнутая конфигурация «нулевого уровня утечек» является важным показателем качества, особенно для систем, работающих длительное время без перезагрузки.

Тестирование и отладка

Тестирование проводилось в два этапа. Сначала выполнялось модульное тестирование CRUD-операций с помощью специально написанной программы test_database.c. Сценарий проверяет:

- корректное добавление и последующий поиск записей;
- обновление полей существующей записи;
- логическое удаление и отсутствие удалённой записи в результатах поиска и списке;
- поведение на граничных условиях (поиск несуществующего ID, работа с пустой базой).

На втором этапе проверялась работа потокового анализатора: утилита generate создавала файл размером более 10 миллионов строк, после чего stream_stats обрабатывал его, а корректность результата сверялась с эталонными значениями, вычисленными альтернативным способом.

Для контроля утечек памяти все исполняемые файлы запускались под Valgrind с флагами --leak-check=full --show-leak-kinds=all. Выявленные на ранних итерациях ошибки, связанные с незакрытыми файловыми дескрипторами и отсутствием сброса буферов, были оперативно устранены. Итоговый вердикт Valgrind: All heap blocks were freed — no leaks are possible.

Результаты внедрения

Результатом работы стал набор консольных утилит, готовых к использованию:

`database_app` — управление файловой базой данных с полным набором CRUD-операций;

`generate` — генерация текстовых файлов произвольной длины с числами;

`stream_stats` — потоковое вычисление статистик по большому файлу с сохранением результата на диск.

Все компоненты собираются единой командой `make`, проходят автоматическое тестирование и не требуют внешних библиотек. Потребление памяти утилитой `stream_stats` остаётся в пределах нескольких мегабайт независимо от размера входных данных. Файловая база данных использует бинарный формат фиксированной длины, что обеспечивает предсказуемую скорость доступа и простоту восстановления.

Выводы

В ходе практики разработана файловая база данных на языке C с фиксированной бинарной структурой записей, реализующая полный цикл CRUD-операций и использующая логическое удаление вместо физического для сохранения целостности файла. Созданы утилита-генератор тестовых данных и программа потоковой агрегации числовых файлов, которая обрабатывает миллионы строк с фиксированным объёмом оперативной памяти, что демонстрирует подход к работе с данными, превышающими доступную память. Проект организован по модульному принципу с вынесением интерфейса в заголовочный файл, сборка автоматизирована Makefile, а проверка Valgrind подтвердила отсутствие утечек памяти во всех исполняемых компонентах. В результате приобретены практические навыки низкоуровневого управления файловым вводом-выводом, построения переносимых системных утилит без внешних зависимостей и инструментального контроля качества кода.

Список использованных источников

1. Kernighan B.W., Ritchie D.M. The C Programming Language. 2nd ed. Prentice Hall, 1988.
2. Pro Git. Scott Chacon, Ben Straub. Apress, 2014.
3. Официальная документация GCC: <https://gcc.gnu.org/manual/>
4. Документация по Valgrind: <http://valgrind.org/docs/>